

For both set of questions paper:

1. Hdfs architecture (20 marks)
2. Hadoop ecosystem with real-time example (20 marks)
3. Apach spark architecture with real-time example (20 marks)
4. Explain about anatomy in hdfs (20 marks)
5. Hive databse with all queries and real-time example (20 marks)
6. Explain about yarn (20 marks)

Practical :

1. Cloudera commands - 10 marks
2. Apach spark python code - 10 marks

1. Hdfs architecture

HDFS (Hadoop Distributed File System) is a distributed file system designed to store and manage large volumes of data across multiple machines in a Hadoop cluster. It is optimized for high-throughput access and fault tolerance. Below is an overview of its architecture:

Key Components of HDFS Architecture

1. NameNode (Master Node):

- **Acts as the central metadata manager.**
- **Stores metadata about the file system, including:**
 - **File and directory structure.**
 - **Block locations.**
 - **File permissions.**
- **Does not store actual data but manages the mapping of files to data blocks.**
- **Maintains the file system namespace and regulates access to files.**
- **Single point of failure in older versions (mitigated by High Availability setups with multiple NameNodes).**

2. DataNodes (Slave Nodes):

- **Store the actual data blocks.**
 - **Perform read/write operations as instructed by the NameNode.**
 - **Periodically send block reports and heartbeats to the NameNode to confirm their status and block health.**
 - **Replicate data blocks to other DataNodes for fault tolerance.**
- 3. Secondary NameNode (Checkpoint Node):**
- **Not a backup NameNode but assists in periodic metadata checkpointing.**
 - **Merges the fsimage (file system metadata snapshot) and edits (log of changes) to reduce NameNode recovery time.**
 - **Does not replace the NameNode in case of failure.**
- 4. Blocks:**
- **HDFS splits files into fixed-size blocks (default size is 128 MB or 256 MB).**
 - **Blocks are distributed across DataNodes for parallel processing and fault tolerance.**
 - **Each block is replicated across multiple DataNodes (default replication factor is 3).**
- 5. Client:**
- **Interacts with the NameNode to perform metadata operations (e.g., file creation, deletion, or opening).**
 - **Communicates directly with DataNodes for reading/writing data.**

HDFS Architecture Workflow

1. File Write:

- **The client contacts the NameNode to request file creation.**
- **The NameNode allocates blocks and provides a list of DataNodes for each block.**
- **The client writes data directly to the DataNodes in a pipeline.**
- **DataNodes replicate the blocks to other nodes as per the replication factor.**

2. File Read:

- **The client requests the NameNode for block locations.**
- **The NameNode returns the addresses of DataNodes holding the required blocks.**
- **The client reads data directly from the DataNodes.**

3. Fault Tolerance:

- If a DataNode fails, the NameNode detects it through missing heartbeats.
- The NameNode initiates replication of the lost blocks to other DataNodes to maintain the replication factor.

Key Features of HDFS

1. Scalability:

- Can scale to thousands of nodes and store petabytes of data.

2. Fault Tolerance:

- Data replication ensures data availability even if nodes fail.

3. High Throughput:

- Optimized for batch processing and large data sets.

4. Cost-Effective:

- Runs on commodity hardware.

5. Data Locality:

- Moves computation closer to data to reduce network overhead.

HDFS High Availability (HA)

To address the single point of failure in the NameNode:

- **Active and Standby NameNodes:**
 - Two NameNodes run in an active-standby configuration.
 - Shared storage (e.g., Quorum Journal Manager or NFS) ensures metadata consistency.
 - Automatic failover is managed by ZooKeeper.

HDFS Federation

- Allows multiple NameNodes to manage different namespaces.
- Improves scalability and isolation of workloads.

Hadoop ecosystem with real-time example (20 marks)

The **Hadoop ecosystem** is a collection of open-source tools and frameworks designed to handle big data processing, storage, and analysis. It is widely used for distributed computing and is particularly effective for processing large datasets across clusters of computers. Below is an explanation of the Hadoop ecosystem with a **real-time example** to illustrate its components and functionality.

Components of the Hadoop Ecosystem

1. HDFS (Hadoop Distributed File System):

- A distributed file system that stores data across multiple machines.
- Provides high throughput and fault tolerance.

2. MapReduce:

- A programming model for processing large datasets in parallel across a distributed cluster.
- Divides tasks into "map" and "reduce" phases.

3. YARN (Yet Another Resource Negotiator):

- Manages resources in the cluster and schedules tasks.
- Acts as the operating system for Hadoop.

4. Hive:

- A data warehousing tool that provides SQL-like queries (HiveQL) to analyze data stored in HDFS.

5. Pig:

- A high-level scripting language (Pig Latin) for data processing.
- Simplifies the creation of MapReduce jobs.

6. HBase:

- A NoSQL database that provides real-time read/write access to large datasets.

7. Spark:

- An in-memory processing engine for real-time data processing and analytics.
- Faster than MapReduce for iterative tasks.

8. Kafka:

- A distributed streaming platform for real-time data pipelines.

9. **Flume:**

- A tool for collecting, aggregating, and moving large amounts of log data into HDFS.

10. **Sqoop:**

- A tool for transferring data between Hadoop and relational databases.

11. **Oozie:**

- A workflow scheduler to manage Hadoop jobs.

12. **Zookeeper:**

- A coordination service for distributed applications.

Real-Time Example: E-Commerce Analytics

Let's consider an **e-commerce platform** like Amazon or Flipkart. The platform generates massive amounts of data from user activities, such as:

- Product searches
- Purchases
- Reviews
- Clickstream data
- Inventory updates

How Hadoop Ecosystem is Used

1. **Data Ingestion:**

- **Kafka** and **Flume** are used to collect real-time data streams from user activities and logs.
- Example: User clicks on a product are captured and sent to Kafka for real-time processing.

2. **Data Storage:**

- The raw data is stored in **HDFS** for scalable and fault-tolerant storage.
- Example: All user activity logs are stored in HDFS for future analysis.

3. **Data Processing:**

- **Spark** processes real-time data streams to generate insights.
 - Example: Real-time recommendations are generated based on user behavior.
- **MapReduce** is used for batch processing of historical data.
 - Example: Analyzing sales trends over the past year.

4. Data Querying:

- **Hive** is used to run SQL-like queries on the data stored in HDFS.
 - Example: Querying the most popular products in a specific category.

5. Real-Time Database:

- **HBase** is used for real-time read/write operations.
 - Example: Updating inventory levels in real-time as products are purchased.

6. Data Integration:

- **Sqoop** transfers data from transactional databases (e.g., MySQL) to HDFS for analysis.
 - Example: Importing customer details from a relational database into Hadoop.

7. Workflow Management:

- **Oozie** schedules and manages workflows for data processing tasks.
 - Example: Automating the daily ETL (Extract, Transform, Load) process.

8. Coordination:

- **Zookeeper** ensures coordination between distributed services.
 - Example: Managing leader election and configuration for Kafka brokers.

Benefits of Using Hadoop Ecosystem

1. **Scalability:** Handles petabytes of data across thousands of nodes.
2. **Fault Tolerance:** Automatically recovers from hardware failures.
3. **Cost-Effective:** Uses commodity hardware.
4. **Flexibility:** Supports structured, semi-structured, and unstructured data.
5. **Real-Time Processing:** Tools like Spark and Kafka enable real-time analytics.

Conclusion

The Hadoop ecosystem is a powerful suite of tools for big data processing and analytics. In the e-commerce example, it enables real-time recommendations, inventory management, and trend analysis, helping businesses make data-driven decisions. By leveraging components like HDFS, Spark, Hive, and Kafka, organizations can efficiently manage and analyze massive datasets.

4. Apach spark architecture with real-time example (20 marks)

Apache Spark Architecture with Real-Time Example

1. Overview of Apache Spark Architecture

Apache Spark is a distributed computing framework designed for fast and flexible data processing. Its architecture is based on a master-slave model, where the **driver program** (master) coordinates the execution of tasks across multiple **worker nodes** (slaves). The key components of Spark architecture are:

1. Driver Program:

- The central point of control in a Spark application.
- Converts the user's code into tasks and schedules them on executors.
- Maintains metadata about the application and its state.

2. Cluster Manager:

- Manages resources across the cluster.
- Examples: Standalone Cluster Manager, Apache Mesos, Hadoop YARN, or Kubernetes.

3. Worker Nodes:

- Machines that run the actual tasks.
- Each worker node has one or more **executors**.

4. Executors:

- Processes that run on worker nodes.
- Execute tasks assigned by the driver and store data in memory or disk.

5. Tasks:

- The smallest unit of work sent to an executor.
- Each task processes a partition of data.

6. RDD (Resilient Distributed Dataset):

- The fundamental data structure in Spark.
- Immutable, distributed collections of objects partitioned across nodes.

7. DAG (Directed Acyclic Graph):

- Spark creates a DAG of operations to optimize task execution.

8. Spark Context:

- The entry point to Spark functionality.
- Coordinates the execution of tasks across the cluster.

2. Real-Time Example: Real-Time Fraud Detection

Let's consider a real-time example of **fraud detection in financial transactions** using Apache Spark Streaming.

Problem Statement:

A bank wants to detect fraudulent transactions in real-time by analyzing transaction patterns. Transactions are streamed in real-time, and the system must flag suspicious transactions based on predefined rules (e.g., high-value transactions, multiple transactions in a short time).

Solution Using Apache Spark:

1. Data Ingestion:

- Transactions are streamed into the system using a messaging system like **Apache Kafka**.
- Spark Streaming consumes this data in micro-batches.

2. Spark Streaming:

- Spark Streaming processes the data in real-time using **DStreams** (Discretized Streams).
- Each micro-batch of transactions is treated as an RDD.

3. Data Processing:

- The transactions are processed using Spark transformations (e.g., map, filter, reduceByKey).
- For example:
 - Filter transactions above a certain threshold (e.g., \$10,000).
 - Group transactions by user ID and count the number of transactions in the last 5 minutes.

4. Fraud Detection Logic:

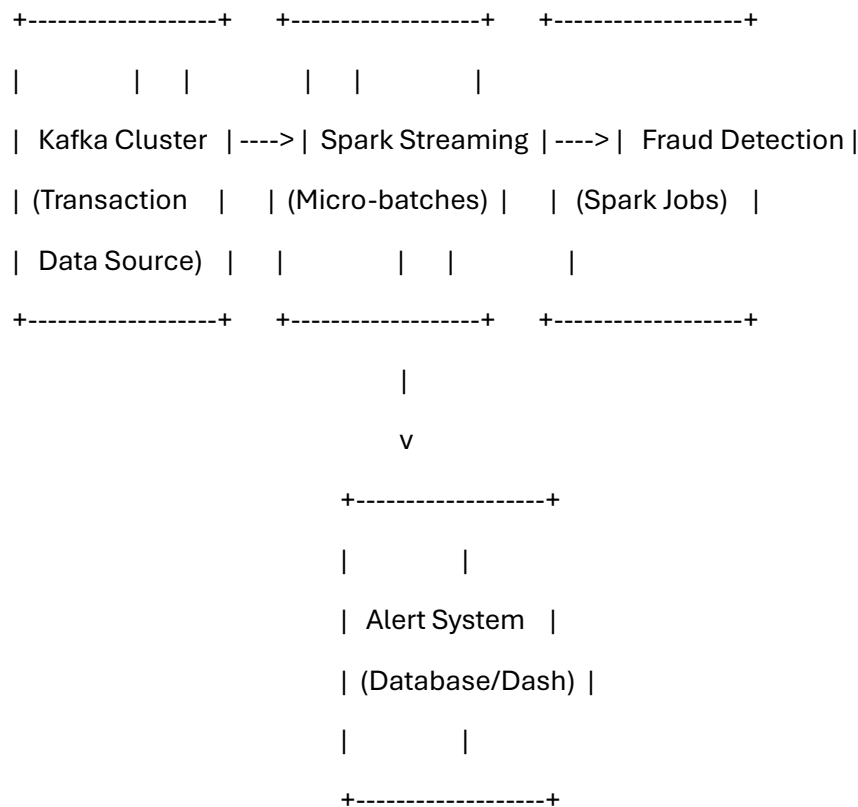
- Apply fraud detection rules:
 - If a user has more than 5 transactions in 5 minutes, flag them as suspicious.
 - If a transaction amount is above \$10,000, flag it for manual review.

5. Output:

- Suspicious transactions are written to a database or alert system for further investigation.
- Results can also be visualized in real-time using a dashboard.

3. Architecture Diagram

Copy



4. Steps in Detail

1. Data Ingestion:

- Transactions are published to Kafka topics by the banking system.
- Spark Streaming subscribes to these topics and ingests data in micro-batches.

2. Stream Processing:

- Spark Streaming creates DStreams from Kafka topics.
- Each DStream is divided into RDDs for parallel processing.

3. Transformations and Actions:

- Apply transformations like map, filter, and reduceByKey to detect fraud.
- Example:

python

Copy

```
suspicious_transactions = transactions.filter(lambda t: t.amount > 10000)
```

4. Output:

- Write flagged transactions to a database or send alerts to a dashboard.

5. Advantages of Using Spark for Real-Time Fraud Detection

1. Low Latency:

- Spark Streaming processes data in near real-time with low latency.

2. Scalability:

- Spark can scale horizontally across a cluster to handle large volumes of transactions.

3. Fault Tolerance:

- Spark ensures fault tolerance through RDD lineage and checkpointing.

4. Ease of Use:

- Spark provides high-level APIs in Java, Scala, Python, and R.

6. Conclusion

Apache Spark's distributed architecture makes it ideal for real-time data processing applications like fraud detection. Its ability to process data in micro-batches, combined with its fault tolerance and scalability, ensures efficient and reliable real-time analytics. In the example above, Spark Streaming enables the bank to detect fraudulent transactions in real-time, reducing financial losses and improving customer trust.

4.Explain about anatomy in hdfs (20 marks)

HDFS (Hadoop Distributed File System) is a distributed file system designed to store and manage large volumes of data across multiple machines in a Hadoop cluster. Its architecture is optimized for handling big data workloads, providing high throughput, fault tolerance, and

scalability. Below is an explanation of the anatomy of HDFS, covering its key components and their roles:

1. Master-Slave Architecture

HDFS follows a master-slave architecture, where a single **NameNode** (master) manages the metadata and coordinates the operations, while multiple **DataNodes** (slaves) store the actual data.

2. Key Components of HDFS

a) NameNode

- The NameNode is the master server in HDFS.
- It manages the file system namespace, including the directory tree and metadata (e.g., file names, permissions, and block locations).
- It does not store the actual data but keeps track of where data blocks are stored across DataNodes.
- It maintains two critical files:
 - **FsImage**: A snapshot of the file system metadata.
 - **EditLog**: A log of all recent changes to the file system.
- The NameNode is a single point of failure (SPOF) in HDFS, though this can be mitigated with High Availability (HA) configurations.

b) DataNode

- DataNodes are the worker nodes that store the actual data blocks.
- They perform read and write operations as instructed by the NameNode.
- DataNodes periodically send **heartbeats** and **block reports** to the NameNode to confirm their availability and report the status of the data blocks they store.

c) Secondary NameNode

- The Secondary NameNode is not a backup for the NameNode but assists in maintaining the file system metadata.
- It periodically merges the **FsImage** and **EditLog** to create a new checkpoint, reducing the NameNode's recovery time in case of failure.

3. Data Storage in HDFS

a) Blocks

- Files in HDFS are divided into fixed-size blocks (default size is 128 MB or 256 MB).

- These blocks are distributed across multiple DataNodes in the cluster.
- Large block sizes reduce the overhead of managing metadata and improve throughput for large files.

b) Replication

- HDFS ensures fault tolerance by replicating each block across multiple DataNodes (default replication factor is 3).
- Replication provides data redundancy, ensuring data availability even if a DataNode fails.

4. File Write and Read Operations

a) File Write

1. The client contacts the NameNode to request permission to write a file.
2. The NameNode checks permissions and provides a list of DataNodes where the blocks should be written.
3. The client writes the data blocks directly to the DataNodes in a pipeline.
4. DataNodes replicate the blocks to other nodes as per the replication factor.

b) File Read

1. The client contacts the NameNode to get the locations of the blocks for the requested file.
2. The NameNode returns the list of DataNodes holding the blocks.
3. The client reads the blocks directly from the DataNodes in parallel.

5. Fault Tolerance and Recovery

- HDFS is designed to handle hardware failures gracefully.
- If a DataNode fails, the NameNode detects the failure through missing heartbeats and replicates the lost blocks from other nodes.
- In High Availability (HA) setups, a standby NameNode can take over if the active NameNode fails.

6. Rack Awareness

- HDFS is rack-aware, meaning it knows the physical location of DataNodes in the cluster.
 - It places block replicas on different racks to ensure data availability even if an entire rack fails.
-

7. Scalability

- HDFS can scale horizontally by adding more DataNodes to the cluster.
 - The NameNode's metadata is stored in memory, so its capacity is limited by the amount of RAM. However, techniques like Federation can be used to scale the namespace.
-

8. Use Cases

- HDFS is ideal for:
 - Storing and processing large datasets (e.g., logs, sensor data, social media data).
 - Batch processing workloads (e.g., MapReduce jobs).
 - Applications requiring high throughput rather than low latency.
-

Summary

The anatomy of HDFS consists of a master-slave architecture with the NameNode managing metadata and DataNodes storing data blocks. Key features include block storage, replication, fault tolerance, and scalability. HDFS is optimized for handling large-scale data processing workloads in distributed environments.

5.Hive databse with all queries and real-time example (20 marks)

Hive Database with Queries and Real-Time Example

1. Introduction to Hive

Apache Hive is a data warehouse infrastructure built on top of Hadoop for providing data summarization, query, and analysis. It allows users to write SQL-like queries called HiveQL (HQL) to query data stored in Hadoop Distributed File System (HDFS).

2. Key Features of Hive

- **SQL-like Interface:** Hive provides a SQL-like interface called HiveQL.

- **Schema on Read:** Hive uses schema-on-read, meaning the schema is applied when the data is read, not when it is written.
- **Scalability:** Built on Hadoop, Hive can handle large datasets.
- **Extensibility:** Supports custom user-defined functions (UDFs).

3. Hive Database and Tables

- **Database:** A namespace that contains tables.
- **Table:** A collection of rows with a defined schema.

4. Creating a Hive Database

sql

Copy

```
CREATE DATABASE IF NOT EXISTS my_database;
```

```
USE my_database;
```

5. Creating a Hive Table

sql

Copy

```
CREATE TABLE IF NOT EXISTS employee (
```

```
    id INT,
```

```
    name STRING,
```

```
    salary FLOAT,
```

```
    department STRING
```

```
)
```

```
ROW FORMAT DELIMITED
```

```
FIELDS TERMINATED BY ','
```

```
STORED AS TEXTFILE;
```

6. Loading Data into Hive Table

sql

Copy

```
LOAD DATA LOCAL INPATH '/path/to/employee_data.csv' INTO TABLE employee;
```

7. Basic Queries

- **Select Query**

sql

Copy

```
SELECT * FROM employee;
```

- **Filter Query**

sql

Copy

```
SELECT * FROM employee WHERE salary > 50000;
```

- **Aggregate Query**

sql

Copy

```
SELECT department, AVG(salary) AS avg_salary FROM employee GROUP BY department;
```

- **Join Query**

sql

Copy

```
SELECT e.name, d.department_name
```

```
FROM employee e
```

```
JOIN department d
```

```
ON e.department = d.department_id;
```

- **Insert Query**

sql

Copy

```
INSERT INTO TABLE employee VALUES (101, 'John Doe', 60000, 'HR');
```

- **Update Query**

sql

Copy

```
UPDATE employee SET salary = 65000 WHERE id = 101;
```

- **Delete Query**

sql

Copy

```
DELETE FROM employee WHERE id = 101;
```

8. Real-Time Example

Scenario: A company wants to analyze employee data to find the average salary per department and identify employees earning above a certain threshold.

Steps:

1. Create Database and Table

sql

Copy

```
CREATE DATABASE IF NOT EXISTS company;
```

```
USE company;
```

```
CREATE TABLE IF NOT EXISTS employee (
```

```
    id INT,
```

```
    name STRING,
```

```
    salary FLOAT,
```

```
    department STRING
```

```
)
```

```
ROW FORMAT DELIMITED
```

```
FIELDS TERMINATED BY ','
```

```
STORED AS TEXTFILE;
```

2. Load Data

sql

Copy

```
LOAD DATA LOCAL INPATH '/path/to/employee_data.csv' INTO TABLE employee;
```

3. Analyze Data

- Average Salary per Department**

sql

Copy

```
SELECT department, AVG(salary) AS avg_salary FROM employee GROUP BY department;
```

- Employees Earning Above Threshold**

sql

Copy

```
SELECT name, salary FROM employee WHERE salary > 70000;
```


4. Store Results

sql

Copy

```
CREATE TABLE avg_salary_per_dept AS
```

```
SELECT department, AVG(salary) AS avg_salary FROM employee GROUP BY department;
```

9. Conclusion

Hive provides a powerful and scalable way to analyze large datasets using SQL-like queries. It is widely used in big data environments for data warehousing and analytics.

5. Explain about yarn (20 marks)

YARN (Yet Another Resource Negotiator) is a critical component of the Hadoop ecosystem, designed to manage resources and schedule tasks in a distributed computing environment. It plays a pivotal role in enabling Hadoop to process large-scale data efficiently. Below is a detailed explanation of YARN in the context of Big Data analytics:

1. Overview of YARN (5 Marks)

- YARN is the resource management layer of Hadoop 2.x and later versions.
- It separates the responsibilities of resource management and job scheduling/monitoring into distinct components.
- YARN allows Hadoop to support multiple data processing frameworks (e.g., MapReduce, Spark, Hive, etc.) beyond just MapReduce.
- It improves cluster utilization, scalability, and flexibility in handling diverse workloads.

2. Architecture of YARN (5 Marks)

YARN consists of three main components:

1. ResourceManager (RM):

- Acts as the master daemon.
- Manages and allocates resources (CPU, memory) across applications.
- Has two sub-components:

- **Scheduler:** Allocates resources to running applications.
- **ApplicationsManager:** Manages the lifecycle of applications.

2. **NodeManager (NM):**

- Runs on each worker node in the cluster.
- Manages resources (CPU, memory) on individual nodes and reports to the ResourceManager.
- Executes tasks (containers) assigned by the ResourceManager.

3. **ApplicationMaster (AM):**

- One AM is created per application.
- Negotiates resources with the ResourceManager and works with the NodeManager to execute tasks.
- Monitors the progress of tasks and handles failures.

3. **How YARN Works (5 Marks)**

1. **Application Submission:**

- A client submits an application to the ResourceManager.

2. **Resource Allocation:**

- The ResourceManager allocates a container (resources) to start the ApplicationMaster.

3. **Application Execution:**

- The ApplicationMaster negotiates with the ResourceManager for additional resources.
- The ApplicationMaster works with NodeManagers to execute tasks in containers.

4. **Monitoring and Completion:**

- The ApplicationMaster monitors the progress of tasks and reports back to the ResourceManager.
- Once the application is complete, the ApplicationMaster shuts down, and resources are released.

4. **Role of YARN in Big Data Analytics (5 Marks)**

• **Resource Management:**

- Efficiently allocates and manages cluster resources for multiple applications.

• **Multi-Tenancy:**

- Supports multiple data processing frameworks (e.g., MapReduce, Spark, Flink) simultaneously.
 - **Scalability:**
 - Enables Hadoop clusters to scale to thousands of nodes.
 - **Flexibility:**
 - Allows organizations to run diverse workloads (batch, interactive, real-time) on the same cluster.
 - **Fault Tolerance:**
 - Handles node failures and ensures high availability of applications.
-

Conclusion

YARN is a cornerstone of modern Big Data analytics, providing the infrastructure needed to manage resources and execute diverse data processing tasks efficiently. Its architecture and functionality make it an indispensable tool for organizations leveraging Hadoop and other Big Data technologies.